

UNIVERSIDADE FEDERAL DE SERGIPE
CENTRO DE CIÊNCIAS EXATAS E TECNOLOGIA
DEPARTAMENTO DE COMPUTAÇÃO

**SIGEA-GTT: Sistema Inteligente de Gestão de Eventos
Adversos – Global Trigger Tool: Projeto Orientado a Objetos e
Modelagem de Classes UML**

Projeto Orientado a Objetos e Modelagem de Classes

Matheus Araujo Pereira



São Cristóvão – SE

2026

UNIVERSIDADE FEDERAL DE SERGIPE
CENTRO DE CIÊNCIAS EXATAS E TECNOLOGIA
DEPARTAMENTO DE COMPUTAÇÃO

Matheus Araujo Pereira

**SIGEA-GTT: Sistema Inteligente de Gestão de Eventos
Adversos – Global Trigger Tool: Projeto Orientado a Objetos e
Modelagem de Classes UML**

Documento de Projeto Orientado a Objetos e Modelagem de Classes UML submetido ao Departamento de Computação da Universidade Federal de Sergipe como parte integrante do Trabalho de Conclusão de Curso no Curso de Bacharelado em Engenharia de Computação.

Discente: Matheus Araujo Pereira (Matrícula: 202100114080).

Orientador(a): Prof. Dr. Gilton José Ferreira da Silva
Coorientador(a): Prof.^a Dr.^a Ana Waleska Silva Patrício

São Cristóvão – SE

2026

Resumo

O presente documento estabelece a modelagem estrutural e o projeto orientado a objetos do Sistema Inteligente de Gestão de Eventos Adversos baseado no método *Global Trigger Tool* (SIGEA-GTT), desenvolvido na Universidade Federal de Sergipe (UFS). Fundamentado nos preceitos formais da especificação OMG UML 2.5.1, nos princípios de arquitetura de software SOLID e no catálogo de padrões de projeto corporativos e do GoF (*Gang of Four*), o trabalho documenta exaustivamente todas as classes que estruturam o backend da aplicação em Java 21 e Spring Boot 3.3, bem como o modelo reativo do frontend em TypeScript e Angular 18+. O documento decompõe o sistema em camadas de software rigorosamente desacopladas: a Camada de Domínio, modelando as entidades JPA centrais (User, GttModule, GttTrigger, HarmSeverity, AcademicClass, Activity e ActivitySubmission) e seus respectivos objetos de valor embutidos e semiestruturados em JSONB; a Camada de Persistência, formalizando as interfaces de repositório baseadas em Spring Data JPA; a Camada de Serviços, encapsulando as regras de negócio clínicas, o controle transacional ACID e as validações institucionais de domínio exclusivo (@academico.ufs.br); a Camada de Apresentação REST, detalhando os controladores RESTful, o padrão *Data Transfer Object* (DTO), mappers estruturais (MapStruct) e manipuladores globais de exceção; e a modelagem estrutural do frontend, ancorada em *Standalone Components*, serviços singleton e gestão reativa de estado com *Angular Signals*. São apresentados diagramas de classes em notação gráfica vetorial nativa TikZ para cada subsistema e camada arquitetural, acompanhados por dicionários descritivos de atributos, métodos, visibilidades e multiplicidades. Por fim, são consolidadas matrizes de rastreabilidade bidirecional mapeando as classes aos vinte e cinco requisitos funcionais do sistema, aos casos de uso, às tabelas físicas do banco de dados relacional e aos padrões de projeto aplicados, atestando a robustez estrutural, a alta coesão e o baixo acoplamento da solução.

Palavras-chave: Engenharia de Software. Diagrama de Classes. UML 2.5. Programação Orientada a Objetos. Spring Boot. Angular. SIGEA-GTT. DCOMP/UFS.

Abstract

This document establishes the structural modeling and object-oriented design of the Intelligent Adverse Event Management System based on the Global Trigger Tool methodology (SIGEA-GTT), developed at the Federal University of Sergipe (UFS). Grounded in the formal precepts of the OMG UML 2.5.1 specification, SOLID software architecture principles, and the catalog of Gang of Four (GoF) and enterprise application design patterns, this work comprehensively documents all classes structuring the backend application in Java 21 and Spring Boot 3.3, as well as the reactive frontend architecture in TypeScript and Angular 18+. The document decomposes the system into strictly decoupled architectural layers: the Domain Layer, modeling the core JPA entities (User, GttModule, GttTrigger, HarmSeverity, AcademicClass, Activity, and ActivitySubmission) and their embedded/semi-structured JSONB value objects; the Persistence Layer, formalizing repository interfaces derived from Spring Data JPA; the Service Layer, encapsulating clinical business logic, ACID transaction management, and institutional domain validation (@academico.ufs.br); the REST Presentation Layer, detailing RESTful controllers, the Data Transfer Object (DTO) pattern, structural MapStruct mappers, and global exception handlers; and the frontend structural model, centered on Standalone Components, singleton services, and reactive state management via Angular Signals. Class diagrams are presented in native vector TikZ notation for each subsystem and architectural layer, accompanied by exhaustive descriptive dictionaries detailing attributes, methods, visibility markers, and multiplicities. Finally, bidirectional traceability matrices map the classes to all twenty-five functional requirements, use cases, physical relational database tables, and applied design patterns, demonstrating architectural robustness, high cohesion, and low coupling.

Keywords: Software Engineering. Class Diagram. UML 2.5. Object-Oriented Programming. Spring Boot. Angular. SIGEA-GTT. DCOMP/UFS.

Lista de ilustrações

Figura 1 – Diagrama de Classes de Domínio do SIGEA-GTT (Entidades JPA)	15
Figura 2 – Diagrama de Classes de Repositórios (Spring Data JPA)	20
Figura 3 – Diagrama de Classes da Camada de Serviços (Spring Services)	23
Figura 4 – Diagrama de Classes da Camada de Apresentação REST e DTOs	26
Figura 5 – Diagrama de Classes e Componentes do Frontend (Angular 18+)	28

Lista de tabelas

Tabela 1 – Estratificação Arquitetural das Classes do SIGEA-GTT	13
Tabela 2 – Matriz de Rastreabilidade entre Classes de Software e Requisitos Funcionais	31
Tabela 3 – Matriz de Rastreabilidade entre Classes e Casos de Uso	32
Tabela 4 – Catálogo de Padrões de Projeto Aplicados no SIGEA-GTT	32

Lista de abreviaturas e siglas

5W2H	<i>What, Why, Where, When, Who, How, How Much</i> (Metodologia de Plano de Ação)
ABNT	Associação Brasileira de Normas Técnicas
ACID	Atomicidade, Consistência, Isolamento e Durabilidade (<i>Atomicity, Consistency, Isolation, Durability</i>)
API	Interface de Programação de Aplicações (<i>Application Programming Interface</i>)
CBO	Acoplamento entre Objetos (<i>Coupling Between Objects</i>)
CCET	Centro de Ciências Exatas e Tecnologia da UFS
CRUD	Criar, Recuperar, Atualizar e Deletar (<i>Create, Read, Update, Delete</i>)
DAO	Objeto de Acesso a Dados (<i>Data Access Object</i>)
DCOMP	Departamento de Computação da Universidade Federal de Sergipe
DDD	Projeto Orientado a Domínio (<i>Domain-Driven Design</i>)
DI	Injeção de Dependências (<i>Dependency Injection</i>)
DTO	Objeto de Transferência de Dados (<i>Data Transfer Object</i>)
EA	Evento Adverso
EHR	Prontuário Eletrônico do Paciente (<i>Electronic Health Record</i>)
GIN	Índice Invertido Generalizado (<i>Generalized Inverted Index</i>)
GoF	<i>Gang of Four</i> (Autores clássicos dos padrões de projeto de software)
GUT	Gravidade, Urgência e Tendência (Matriz de Priorização)
GTT	<i>Global Trigger Tool</i> (Rastreador Global de Gatilhos)
HD	Alta Definição (<i>High Definition</i> – 1280 × 720)
HTTP	Protocolo de Transferência de Hipertexto (<i>Hypertext Transfer Protocol</i>)
HU-UFS	Hospital Universitário da Universidade Federal de Sergipe
IEEE	<i>Institute of Electrical and Electronics Engineers</i>

IHI	<i>Institute for Healthcare Improvement</i>
IoC	Inversão de Controle (<i>Inversion of Control</i>)
ISO	<i>International Organization for Standardization</i>
JPA	<i>Java Persistence API</i>
JSON	Notação de Objetos JavaScript (<i>JavaScript Object Notation</i>)
JSONB	Formato Binário Decomposto de JSON (<i>Binary JavaScript Object Notation</i>)
JWT	<i>JSON Web Token</i>
LCOM	Falta de Coesão nos Métodos (<i>Lack of Cohesion in Methods</i>)
LGPD	Lei Geral de Proteção de Dados Pessoais (Lei Federal nº 13.709/2018)
NCC MERP	<i>National Coordinating Council for Medication Error Reporting and Prevention</i>
OMG	<i>Object Management Group</i>
ORM	Mapeamento Objeto-Relacional (<i>Object-Relational Mapping</i>)
PDCA	<i>Plan, Do, Check, Act</i> (Ciclo de Deming para Melhoria Contínua)
POO	Programação Orientada a Objetos
RBAC	Controle de Acesso Baseado em Papéis (<i>Role-Based Access Control</i>)
REST	Transferência de Estado Representacional (<i>Representational State Transfer</i>)
RF	Requisito Funcional
RNF	Requisito Não Funcional
SGBD	Sistema de Gerenciamento de Banco de Dados
SIGEA-GTT	Sistema Inteligente de Gestão de Eventos Adversos – Global Trigger Tool
SOLID	Princípios de Design de Software (<i>Single Responsibility, Open/Closed, Liskov, Interface Segregation, Dependency Inversion</i>)
UFS	Universidade Federal de Sergipe
UML	Linguagem de Modelagem Unificada (<i>Unified Modeling Language</i>)
UUID	Identificador Único Universal (<i>Universally Unique Identifier</i>)
WUXGA	<i>Wide Ultra Extended Graphics Array</i> (1920 × 1200)

Sumário

1	Introdução e Fundamentação da Modelagem Orientada a Objetos	10
1.1	Fundamentação da Notação OMG UML 2.5.1	10
1.2	Princípios Arquiteturais SOLID	12
1.3	Catálogo de Padrões de Projeto Aplicados	12
1.4	Visão Geral da Arquitetura em Camadas	13
2	Modelo de Domínio e Entidades de Negócio (Domain Layer)	14
2.1	Diagrama de Classes de Domínio	14
2.2	Detalhamento das Entidades JPA de Negócio	14
2.2.1	Entidade: User	14
2.2.2	Entidade: AcademicClass	16
2.2.3	Entidades: GttModule, GttTrigger e HarmSeverity	16
2.2.4	Entidades: Activity e ActivitySubmission	17
2.3	Objetos de Valor e Modelagem de Dados Clínicos em JSONB	17
3	Camada de Acesso a Dados e Persistência (Repository Layer)	19
3.1	Diagrama de Classes da Camada de Repositórios	19
3.2	Detalhamento dos Contratos de Persistência	19
3.3	Políticas Transacionais e Concorrência	21
4	Camada de Serviços e Regras de Negócio (Service Layer)	22
4.1	Diagrama de Classes da Camada de Serviços	22
4.2	Detalhamento das Regras de Negócio Invariantes	22
5	Camada de Apresentação REST e Transferência de Dados (Controller & DTO Layer)	25
5.1	Diagrama de Classes da Camada de Apresentação REST	25
5.2	Padrão DTO e Mapeamento Estrutural com MapStruct	25
5.3	Tratamento Global de Exceções e Resposta Padronizada	26
6	Modelagem Estrutural do Frontend (Angular 18+ e Signals)	27
6.1	Diagrama de Classes e Componentes do Frontend	27
6.2	Gerenciamento Reativo de Estado com Angular Signals	27
6.3	Segurança de Rotas e Interceptores HTTP	28
7	Matrizes de Rastreabilidade Estrutural e Arquitetural	30
7.1	Matriz de Rastreabilidade: Classes vs. Requisitos Funcionais	30

7.2	Matriz de Rastreabilidade: Classes vs. Casos de Uso	30
7.3	Matriz de Padrões de Projeto Aplicados	30
8	Verificação, Validação e Considerações Finais	33
8.1	Análise de Métricas Estruturais Orientadas a Objetos	33
8.2	Cumprimento do Critério de Aceite Estrito (<i>Quality Gate</i>)	34
8.3	Considerações Finais	34
	Referências	35

1

Introdução e Fundamentação da Modelagem Orientada a Objetos

A Engenharia de Software contemporânea orientada a domínios complexos exige modelos estruturais de alta fidelidade semântica, capazes de capturar tanto as restrições invariantes de negócio quanto a organização modular do código-fonte ([BOOCH et al., 2007](#); [EVANS, 2003](#)). No desenvolvimento do ****SIGEA-GTT**** (Sistema Inteligente de Gestão de Eventos Adversos baseado na metodologia *Global Trigger Tool*), a complexidade clínica da auditoria hospitalar — caracterizada pela coexistência de taxonomias internacionais consolidadas ([GRIFFIN; RESAR, 2009](#); [NATIONAL COORDINATING COUNCIL FOR MEDICATION ERROR REPORTING AND PREVENTION, 2001](#)), prontuários semiestruturados, restrições temporais estritas de vinte minutos e instrumentos de causa-raiz da qualidade — demanda uma engenharia orientada a objetos rigorosamente fundamentada.

O Diagrama de Classes constitui o principal artefato estrutural da *Unified Modeling Language* (UML), padronizada pela *Object Management Group* ([OBJECT MANAGEMENT GROUP, 2017](#)). Este documento formaliza o projeto estático do sistema, detalhando a hierarquia de tipos, associações, multiplicidades, contratos de interface e padrões de projeto implementados no backend em ****Java 21 / Spring Boot 3.3**** e no frontend reativo em ****TypeScript / Angular 18+****.

1.1 Fundamentação da Notação OMG UML 2.5.1

Conforme a especificação formal OMG UML 2.5.1 ([OBJECT MANAGEMENT GROUP, 2017](#)) e a literatura canônica de modelagem ([FOWLER, 2003](#)), uma classe é um classificador estrutural que descreve um conjunto compartilhado de atributos, operações, restrições e relacionamentos. Neste documento, as classes são formalizadas por meio de compartimentos estanques:

1. **Compartimento de Identificação:** Contém o nome unívoco da classe e seus estereótipos (ex.: «Entity», «Service», «Repository», «RestController»);
2. **Compartimento de Atributos:** Descreve os dados internos e variáveis de instância no formato padrão visibilidade nome : tipo [multiplicidade] = valor inicial;
3. **Compartimento de Operações:** Especifica os métodos e comportamentos disponibilizados pela classe na sintaxe visibilidade nome(parâmetros) : tipo de retorno.

A visibilidade dos membros obedece rigorosamente aos modificadores canônicos da UML:

- **Público (+):** Acessível por qualquer classe cliente que possua visibilidade sobre o classificador;
- **Privado (-):** Restrito exclusivamente ao escopo interno da própria classe definidora;
- **Protegido (#):** Acessível pela própria classe e por suas subclasses na hierarquia de generalização;
- **Pacote / Padrão (~):** Acessível apenas por classes pertencentes ao mesmo pacote lógico.

Os relacionamentos estáticos entre as classes são categorizados da seguinte forma:

- **Associação Simples:** Conexão estrutural semântica bidirecional ou unidirecional (->) com indicação de navegabilidade e multiplicidades nas extremidades (1, 0..1, *, 1..*);
- **Agregação:** Associação do tipo todo-parte fraca, representada por um losango vazado (<>-);
- **Composição:** Associação forte do tipo todo-parte com dependência existencial estrita da parte em relação ao todo, representada por um losango preenchido (<*>-);
- **Generalização / Herança:** Mecanismo de extensão polimórfica estrutural, denotado por uma seta com triângulo vazado (-|>);
- **Realização de Interface:** Contrato formal de cumprimento de operações por uma classe concreta, denotado por linha tracejada com triângulo vazado (. .|>);
- **Dependência:** Relação de uso transitório onde a alteração de uma classe fornecedora afeta a cliente, representada por linha tracejada direcionada (. .>).

1.2 Princípios Arquiteturais SOLID

Para assegurar manutenibilidade, extensibilidade e testabilidade plena com 100% de cobertura, a modelagem de classes do SIGEA-GTT é regida pelos cinco princípios ****SOLID**** formalizados por Robert C. Martin ([MARTIN, 2002](#)):

1. **Single Responsibility Principle (SRP)**: Cada classe possui exatamente uma única razão para mudar. No SIGEA-GTT, as entidades de domínio JPA apenas modelam estado e relacionamentos; as classes de repositório apenas abstraem persistência; os serviços contêm apenas regras de negócio; e os controladores REST apenas tratam o protocolo HTTP;
2. **Open/Closed Principle (OCP)**: O software é aberto para extensão, mas fechado para modificação direta. Mappers estruturais e validadores de domínio foram concebidos de modo que novas regras clínicas possam ser incorporadas por polimorfismo ou novos componentes sem alteração dos núcleos estáveis;
3. **Liskov Substitution Principle (LSP)**: Subtipos devem ser plenamente substituíveis por seus tipos base sem quebrar o comportamento do sistema. As interfaces de persistência herdam de `JpaRepository` garantindo contratos semânticos transparentes;
4. **Interface Segregation Principle (ISP)**: Os clientes não devem ser forçados a depender de métodos que não utilizam. As interfaces do sistema segregam operações de consulta paginada, operações de mutação transacional e consultas analíticas de indicadores;
5. **Dependency Inversion Principle (DIP)**: Módulos de alto nível não dependem de módulos de baixo nível; ambos dependem de abstrações. Na arquitetura do backend, os controladores dependem de interfaces ou abstrações de serviço, e os serviços dependem de abstrações de repositório injetadas pelo container de IoC do Spring Boot.

1.3 Catálogo de Padrões de Projeto Aplicados

O design orientado a objetos do SIGEA-GTT sintetiza soluções consagradas da Engenharia de Software corporativa ([GAMMA et al., 1994](#); [FOWLER, 2002](#)):

- **Repository Pattern**: Encapsula a lógica de acesso a dados e isola a camada de domínio das especificidades do SGBD PostgreSQL 16;
- **Service Layer Pattern**: Define a fronteira da aplicação e estabelece um conjunto de operações disponíveis para os clientes, orquestrando transações atômicas com a anotação `@Transactional`;

- **Data Transfer Object (DTO):** Desacopla as entidades de persistência JPA da camada de apresentação REST, evitando vazamento de atributos internos e otimizando a serialização JSON;
- **Mapper Pattern:** Automatiza a conversão bidirecional entre DTOs e entidades por meio da biblioteca MapStruct, eliminando código repetitivo e garantindo validação em tempo de compilação;
- **Singleton Pattern:** Adotado nativamente pelo Spring Framework para todos os serviços, repositórios e controladores, garantindo que uma única instância compartilhada e sem estado gerencie o ciclo de vida dos componentes;
- **Dependency Injection (DI):** Desacopla a criação e a ligação de dependências, permitindo testes unitários com dublês (Mockito) com cobertura integral de branches;
- **Chain of Responsibility / Interceptor Filter:** Implementado no pipeline de segurança (JwtAuthenticationFilter) para interceptar requisições HTTP, validar tokens criptográficos e popular o contexto de segurança de forma transparente.

1.4 Visão Geral da Arquitetura em Camadas

A Tabela 1 resume a estratificação arquitetural de classes adotada no SIGEA-GTT:

Tabela 1 – Estratificação Arquitetural das Classes do SIGEA-GTT

Camada de Software	Tecnologia Alvo	Estereótipo	Papel e Responsabilidade Estrutural
Domínio (Domain)	Java 21 / JPA	«Entity»	Modela entidades de negócio, invariantes, chaves primárias UUID e colunas semiestruturadas.
Persistência (Repo)	Spring Data JPA	«Repository»	Interfaces de acesso a dados com métodos derivados e consultas JPQL especializadas.
Negócio (Service)	Spring Service	«Service»	Orquestração de regras clínicas, controle transacional ACID e validações institucionais.
Apresentação REST	Spring Web MVC	«RestController»	Endpoints RESTful, negociação de conteúdo HTTP, DTOs e mappers estruturais.
Frontend Reativo	Angular 18+ / TS	«Component»	Standalone components, gerenciamento de estado reativo com Signals e serviços singleton.

Fonte: Elaborado pelo autor (2026).

2

Modelo de Domínio e Entidades de Negócio (Domain Layer)

A camada de domínio concentra a inteligência de negócio, as regras clínicas da metodologia IHI-GTT e as restrições invariantes do SIGEA-GTT. Projetada sob os preceitos do *Domain-Driven Design* (DDD) (EVANS, 2003), esta camada é livre de dependências de infraestrutura externa ou protocolos de transporte, sendo composta por Entidades JPA fortemente tipadas, Enumerações de controle de acesso e Objetos de Valor (*Value Objects*) que modelam documentos clínicos semiestruturados.

2.1 Diagrama de Classes de Domínio

A Figura 1 apresenta o Diagrama Geral de Classes de Domínio do SIGEA-GTT, renderizado em notação gráfica nativa TikZ com fidelidade à especificação OMG UML 2.5.1:

2.2 Detalhamento das Entidades JPA de Negócio

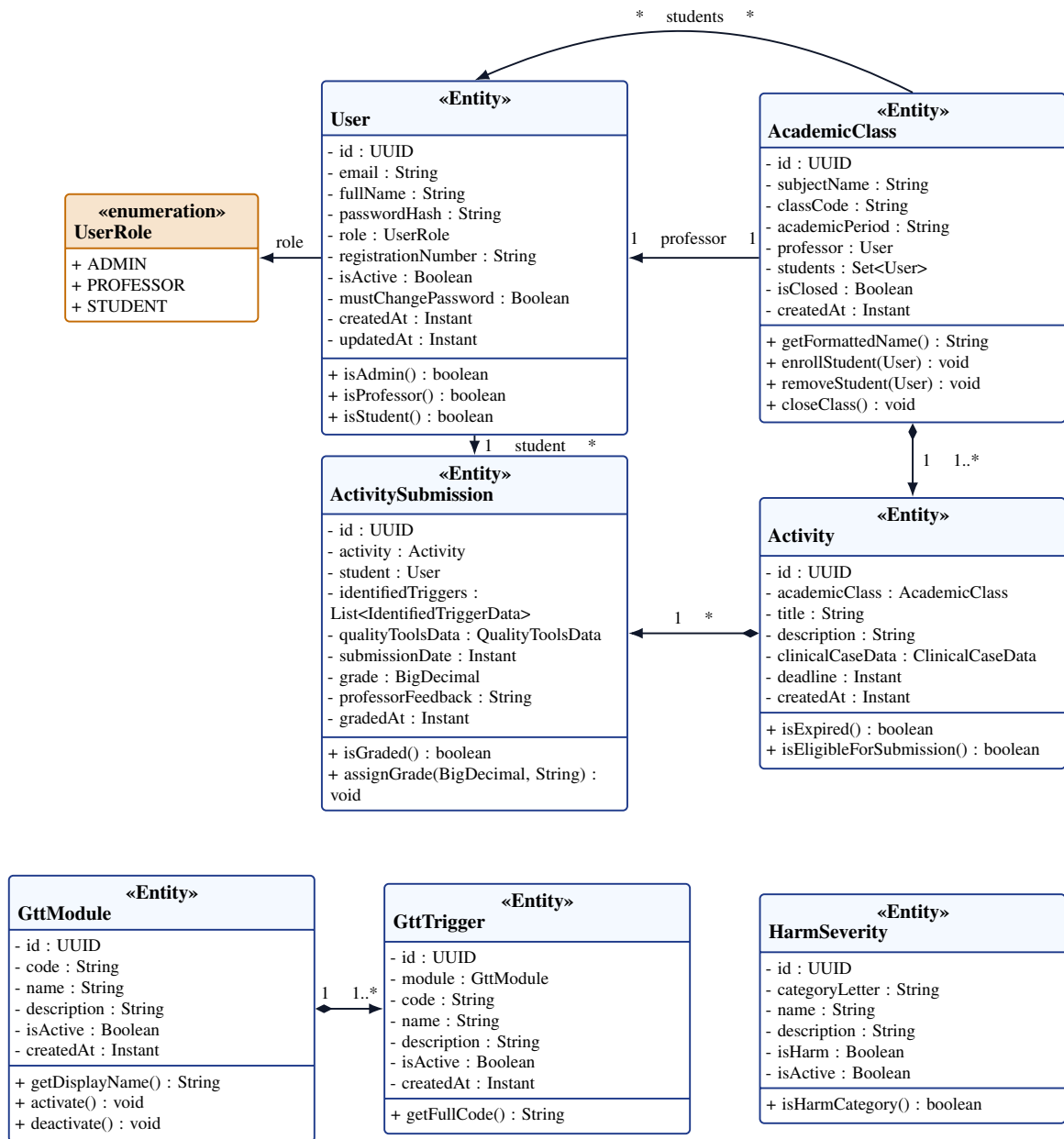
A seguir são descritas as especificações formais de atributos, tipos, restrições e comportamentos das entidades mapeadas:

2.2.1 Entidade: User

A classe User modela os atores humanos cadastrados no sistema, implementando a segregação de perfis de acordo com o padrão RBAC:

- `id` (UUID): Chave primária sintética gerada por algoritmo de alta dispersão (UUID v4), impedindo enumeração sequencial maliciosa;
- `email` (String): Endereço institucional obrigatório com constraint de unicidade e validação de domínio estrito (@academico.ufs.br);

Figura 1 – Diagrama de Classes de Domínio do SIGEA-GTT (Entidades JPA)



Fonte: Elaborado pelo autor (2026).

- **fullName (String)**: Nome completo do usuário para exibição em listas e relatórios;
- **passwordHash (String)**: Hash unidirecional salgado gerado pelo algoritmo criptográfico BCrypt com fator de custo 10;
- **role (UserRole)**: Enumeração que categoriza o papel operacional do usuário (ADMIN, PROFESSOR ou STUDENT);
- **registrationNumber (String)**: Matrícula institucional discente. Obrigatória estritamente quando `role == UserRole.STUDENT` e obrigatoriamente nula para docentes e administradores;

- `isActive` (Boolean): Sinalizador booleano de habilitação de acesso, permitindo inativação lógica sem perda de histórico acadêmico;
- `mustChangePassword` (Boolean): Sinalizador de obrigatoriedade de redefinição de senha provisória gerada automaticamente.

2.2.2 Entidade: `AcademicClass`

A classe `AcademicClass` encapsula o agrupamento semestral de discentes sob a supervisão pedagógica de um docente titular:

- `subjectName` (String): Denominação formal da disciplina curricular (ex.: *"Enfermagem na Saúde do Adulto I"*);
- `classCode` (String): Identificador de turma semestral (ex.: *"T01"*);
- `academicPeriod` (String): Período letivo no formato canônico da UFS (ex.: *"2025.2"*);
- `professor` (User): Referência obrigatória (`@NotNull`) ao docente titular da turma;
- `students` (Set<User>): Coleção de discentes matriculados, mapeada pela tabela associativa `class_students`;
- `isClosed` (Boolean): Sinalizador de encerramento acadêmico. Quando verdadeiro, veda novas matrículas e submissões tardias;
- Método `getFormattedName()`: Retorna a denominação concatenada "Disciplina - Turma - Período".

2.2.3 Entidades: `GttModule`, `GttTrigger` e `HarmSeverity`

Estas entidades formam o núcleo taxonômico da metodologia *Global Trigger Tool*:

- `GttModule`: Representa os seis módulos temáticos recomendados pelo IHI ([GRIFFIN; RESAR, 2009](#)): Cuidados Gerais (C), Cirúrgico (S), Medicamentoso (M), Terapia Intensiva (I), Perinatal (P) e Emergência (E);
- `GttTrigger`: Representa os cinquenta e três gatilhos clínicos padronizados. Possui código alfanumérico único (ex.: *"M03"*) e chave estrangeira mandatória para seu respectivo módulo (`ON DELETE RESTRICT`);
- `HarmSeverity`: Representa as nove categorias de dano do NCC MERP ([NATIONAL COORDINATING COUNCIL FOR MEDICATION ERROR REPORTING AND PREVENTION, 2001](#)) (letras 'A' até 'I'). O atributo booleano `isHarm` diferencia incidentes sem dano (`false` para categorias A até D) de eventos adversos com dano real (`true` para categorias E até I).

2.2.4 Entidades: Activity e ActivitySubmission

Estruturam o ciclo de realização e avaliação de auditorias clínicas:

- **Activity**: Modela uma atividade de auditoria designada para uma turma. Contém título, instruções, prazo final (deadline) e o prontuário de alta fidelidade persistido na coluna estruturada `clinicalCaseData`;
- **ActivitySubmission**: Armazena a resolução entregue pelo discente. Agrupa a lista de gatilhos auditados (`identifiedTriggers`), os artefatos de qualidade (`qualityToolsData`), a data de envio, a nota atribuída pelo docente (grade) e o parecer pedagógico formativo registrado.

2.3 Objetos de Valor e Modelagem de Dados Clínicos em JSONB

Para acomodar a dinâmica e a heterogeneidade das informações médicas sem incorrer na rigidez excessiva do modelo relacional puro, o sistema adota Objetos de Valor (*Value Objects*) que são serializados em colunas JSONB do PostgreSQL:

1. **ClinicalCaseData**: Agrega os nós do prontuário simulado:

- **PatientData**: Dados sociodemográficos fictícios (nome, idade, sexo, leito);
- **AdmissionData**: Data de internação, queixa principal e hipótese diagnóstica;
- **List<EvolutionNoteData>**: Evoluções médicas diárias sequenciais;
- **List<PrescriptionData>**: Prescrições farmacológicas ativas e suspensas com dose, via e frequência;
- **List<LabExamData>**: Painéis laboratoriais com resultados numéricos, unidades e faixas de referência;
- **List<ProcedureData>**: Descrições cirúrgicas e procedimentos invasivos.

2. **IdentifiedTriggerData**: Registra os rastros clínicos detectados pelo discente:

- Código e nome do gatilho rastreado;
- Seção do prontuário onde o rastro foi localizado;
- Trecho textual citado da evidência (`quotedEvidence`);
- Justificativa clínica redigida pelo auditor (`clinicalJustification`);
- Categoria de severidade atribuída na escala NCC MERP ('A' a 'I').

3. **QualityToolsData**: Consolida as quatro ferramentas de melhoria contínua:

- `IshikawaData`: Efeito central e causas nos 6 vetores causais (Método, Máquina, Material, Mão de Obra, Medida, Meio Ambiente);
- `List<GutItemData>`: Problemas priorizados com notas de 1 a 5 para Gravidade, Urgência e Tendência, e score *GUT*;
- `List<FiveWTwoHItemData>`: Ações corretivas detalhadas nas dimensões do método 5W2H;
- `PdcaData`: Metas e rotinas nos quatro quadrantes do ciclo PDCA.

3

Camada de Acesso a Dados e Persistência (Repository Layer)

A camada de persistência é responsável por intermediar a comunicação entre o modelo orientado a objetos do SIGEA-GTT e o Sistema de Gerenciamento de Banco de Dados relacional PostgreSQL 16 LTS. Implementada com base no framework Spring Data JPA sobre o mecanismo do Hibernate ORM, esta camada segue rigorosamente o padrão **Repository** (FOWLER, 2002), abstraindo a geração de comandos SQL/DDI e provendo contratos de consulta fortemente tipados.

3.1 Diagrama de Classes da Camada de Repositórios

A Figura 2 ilustra a hierarquia estrutural de interfaces que compõem a camada de acesso a dados, com ênfase na herança de `JpaRepository<T, UUID>` e na especificação das operações de busca especializadas:

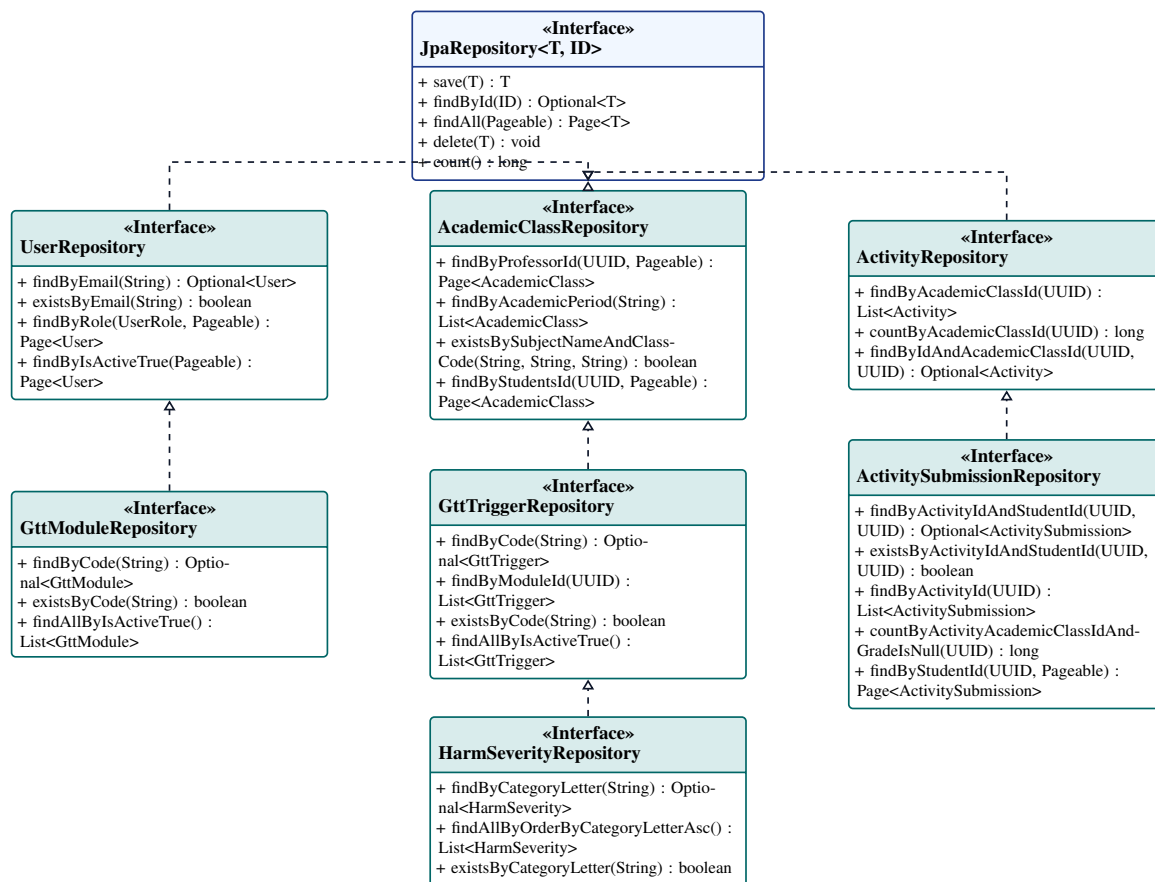
3.2 Detalhamento dos Contratos de Persistência

Cada repositório estende a interface genérica `JpaRepository<T, UUID>`, herdando rotinas de CRUD e paginação e definindo métodos de busca derivada (*Query Methods*) baseados nas convenções do Spring Data:

1. UserRepository:

- `findByEmail(String)`: Busca atômica por e-mail institucional, utilizada criticamente no fluxo de autenticação JWT;
- `existsByEmail(String)`: Verificação rápida de colisão cadastral antes de persistência;

Figura 2 – Diagrama de Classes de Repositórios (Spring Data JPA)



Fonte: Elaborado pelo autor (2026).

- `findByRole(UserRole, Pageable)`: Listagem paginada segregando discentes, docentes e administradores com exatamente dez itens por página.

2. AcademicClassRepository:

- `findByProfessorId(UUID, Pageable)`: Filtra as turmas atribuídas a um professor titular específico;
- `existsBySubjectNameAndClassCodeAndAcademicPeriod(...)`: Valida a regra de unicidade composta para impedir duplicidade de turmas no mesmo período letivo semestral;
- `findByStudentsId(UUID, Pageable)`: Recupera as turmas nas quais determinado aluno possui matrícula ativa.

3. ActivitySubmissionRepository:

- `findByActivityIdAndStudentId(UUID, UUID)`: Localiza a submissão de um discente em determinada atividade, garantindo a verificação de tentativa única;
- `countByActivityAcademicClassIdAndGradeIsNull(UUID)`: Consulta agregada que contabiliza submissões pendentes de nota na turma,

utilizada como critério de bloqueio para o encerramento do semestre letivo (UC10 / RF13);

- `findByActivityId(UUID)`: Recupera todas as entregas para composição do painel de correção docente.

4. `GttModuleRepository`, `GttTriggerRepository`, `HarmSeverityRepository`:

- `findByCode(String)` / `findByCategoryLetter(String)`: Consultas exatas de alta velocidade aceleradas pelos índices B-Tree criados no banco de dados.

3.3 Políticas Transacionais e Concorrência

A camada de repositório opera em estreita consonância com o gerenciador de transações do Spring Framework (`PlatformTransactionManager`). No SIGEA-GTT, adotam-se as seguintes diretrizes transacionais:

- **Transações Somente-Leitura** (`@Transactional(readOnly = true)`): Aplicadas em todos os métodos de consulta para desabilitar o mecanismo de *dirty checking* do Hibernate, reduzindo o consumo de memória da JVM e acelerando a resposta HTTP;
- **Transações Mutativas Atômicas** (`@Transactional` com `READ_COMMITTED`): Utilizadas em criação de turmas, matrículas e submissões clínicas para assegurar as propriedades ACID e integridade referencial frente a acessos concorrentes na rede.

4

Camada de Serviços e Regras de Negócio (Service Layer)

A camada de serviços (*Service Layer*) constitui a fronteira executiva da aplicação, na qual convergem a coordenação de operações de negócio, o controle de transações ACID e a imposição de regras invariantes institucionais e clínicas. Projetada sob o princípio da Inversão de Dependências (DIP) e gerenciada pelo container de Inversão de Controle (IoC) do Spring Framework, a camada opera de forma totalmente desacoplada de protocolos web ou mecanismos físicos de banco de dados.

4.1 Diagrama de Classes da Camada de Serviços

A Figura 3 detalha as principais classes de serviço do SIGEA-GTT, suas dependências injetadas e as operações de negócio disponibilizadas aos clientes:

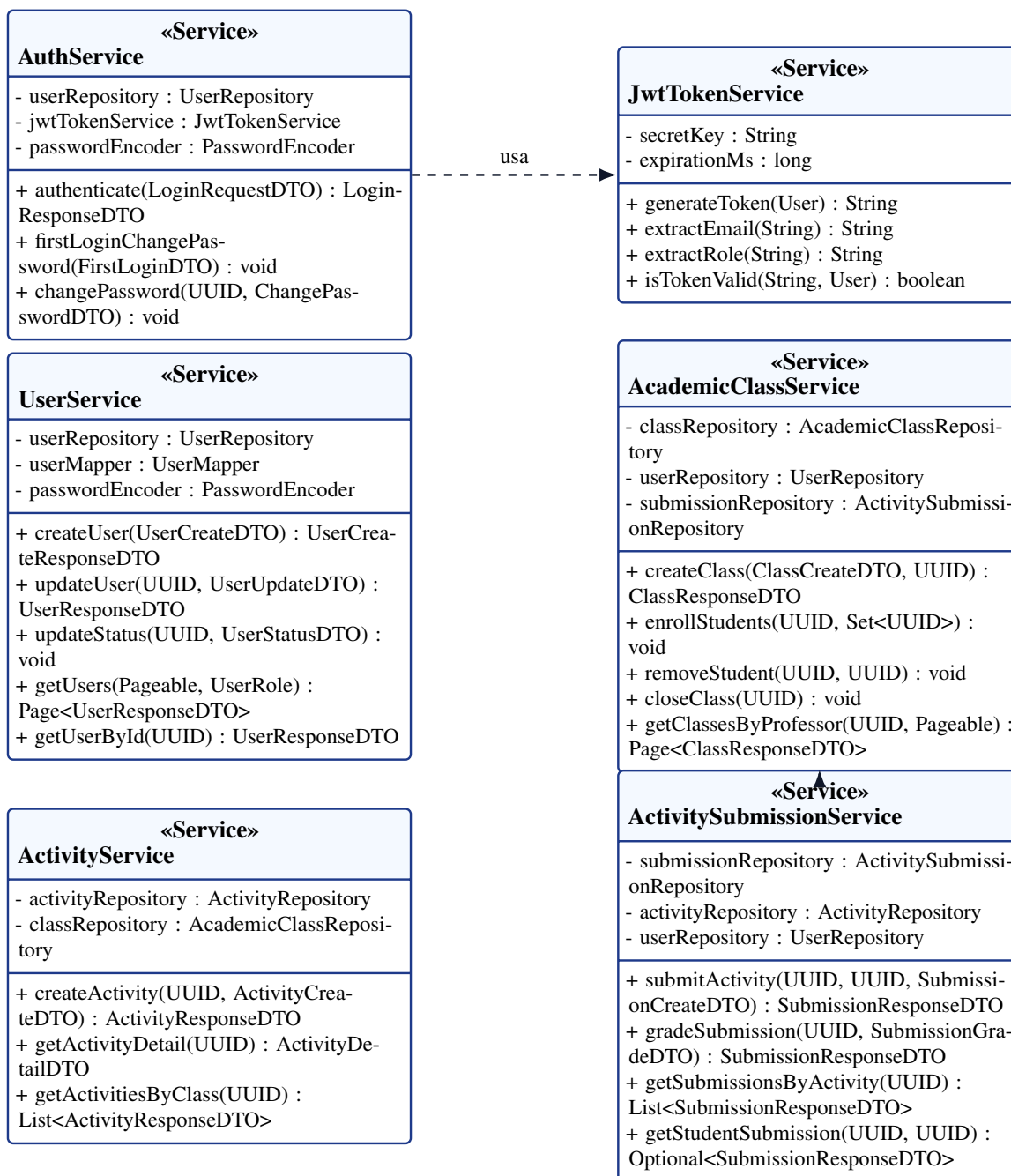
4.2 Detalhamento das Regras de Negócio Invariantes

Os serviços de negócio atuam como guardiões da consistência do sistema, validando fluxos antes de qualquer persistência:

1. Gestão de Identidade e Segurança (AuthService e UserService):

- *Validação de Domínio Institucional*: O método de criação de usuário intercepta o e-mail e aplica o validador @UfsEmail, rejeitando qualquer sufixo diferente de @academico.ufs.br;
- *Segregação de Matrícula Discente*: O serviço checa se o perfil do usuário é discente (STUDENT). Se for discente, a matrícula é obrigatória e deve possuir entre 6 e 30 caracteres alfanuméricos. Se for ADMIN ou PROFESSOR, qualquer tentativa de informar matrícula é forçadamente anulada;

Figura 3 – Diagrama de Classes da Camada de Serviços (Spring Services)



Fonte: Elaborado pelo autor (2026).

- *Fluxo de Senha Provisória*: No primeiro acesso, o método de login verifica se `mustChangePassword == true`. O usuário é impedido de acessar endpoints protegidos até que invoque com sucesso `firstLoginChangePassword`, que atualiza a credencial com BCrypt e define a flag provisória como falsa.

2. Governança Acadêmica de Turmas (AcademicClassService):

- *Padrão Canônico de Nomenclatura*: O serviço concatena e valida a máscara textual universal "Disciplina - Turma - Período" (ex.: "Enfermagem na Saúde do

Adulto I - T01 - 2025.2");

- *Docência Titular Obrigatória:* Toda turma é vinculada ao ID do professor autenticado, impedindo criação de turmas anônimas ou orfãs;
- *Bloqueio de Encerramento com Pendências:* Antes de atualizar `isClosed = true`, o serviço invoca `countByActivityAcademicClassIdAndGradeIsNull(classId)`. Caso existam atividades com submissões não corrigidas pelo professor, lança-se uma `BusinessException` impedindo o encerramento.

3. Auditoria Clínica e Avaliação Docente (ActivitySubmissionService):

- *Verificação de Janela de Entrega:* O serviço verifica se a data/hora atual é estritamente anterior a `activity.getDeadline()`;
- *Submissão Única:* A tentativa de reenvio por discente que já possua registro na atividade é bloqueada;
- *Avaliação com Nota Numérica:* O método de correção exige que a nota esteja no intervalo `[0,00, 10,00]` e que o parecer pedagógico (`professorFeedback`) contenha fundamentação formativa.

4. Cálculo de Indicadores Clínicos IHI (ClassDashboardService):

- Consolida as métricas recomendadas pelo IHI ([GRIFFIN; RESAR, 2009](#)): Taxa de Eventos por 1.000 pacientes-dia, Taxa por 100 admissões e Percentual de internações com dano real (% de categorias E a I).

5

Camada de Apresentação REST e Transferência de Dados (Controller & DTO Layer)

A camada de apresentação REST do SIGEA-GTT é responsável por expor os endpoints da aplicação sob o protocolo HTTP, realizando o roteamento, a desserialização de requisições, a validação declarativa de payloads e a padronização de respostas JSON. Para manter a independência arquitetural preconizada pelo padrão DTO (*Data Transfer Object*) (FOWLER, 2002), nenhuma entidade JPA de persistência é exposta diretamente aos consumidores da API.

5.1 Diagrama de Classes da Camada de Apresentação REST

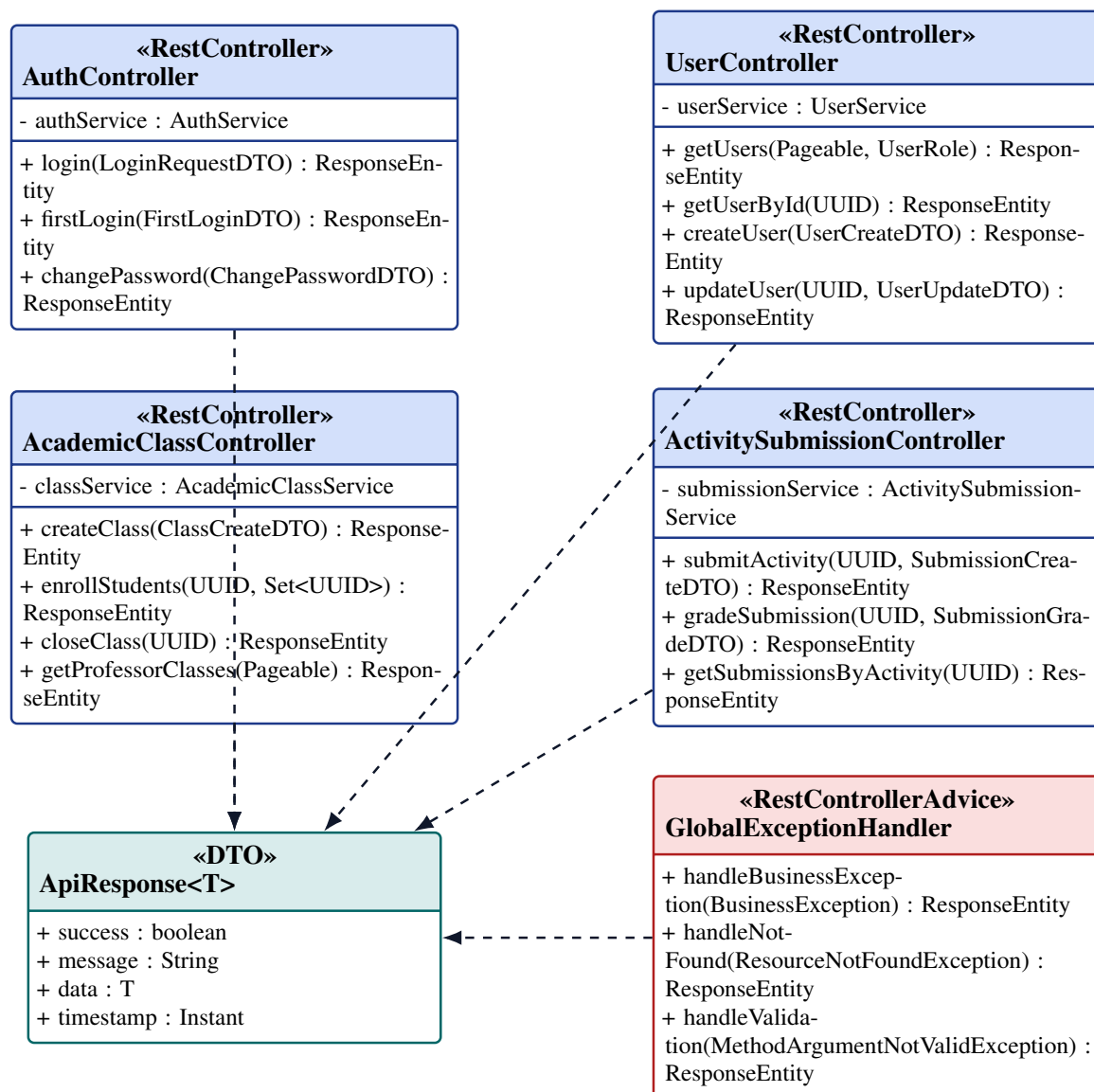
A Figura 4 detalha os controladores RESTful, o tratamento global de exceções e o envelopamento uniforme de respostas:

5.2 Padrão DTO e Mapeamento Estrutural com MapStruct

Para garantir segregação estrita de responsabilidades, os DTOs do SIGEA-GTT são divididos em objetos de comando (*Request DTOs*) e objetos de projeção (*Response DTOs*):

- **DTOs de Entrada (*Request*):** Incorporam anotações do *Bean Validation* (@NotBlank, @Size, @Email, @UfsEmail), garantindo que dados inválidos sejam bloqueados na camada de transporte antes de atingir os serviços;
- **DTOs de Saída (*Response*):** Ocultam campos sensíveis como hashes de senha, projetando apenas atributos necessários para renderização na interface;
- **Mappers Estruturais (MapStruct):** As interfaces UserMapper, GttModuleMapper, GttTriggerMapper e HarmSeverityMapper são processadas em tempo de compilação, gerando código Java puro de alta performance para cópia de propriedades entre entidades e DTOs, evitando o custo computacional de reflexão em tempo de execução.

Figura 4 – Diagrama de Classes da Camada de Apresentação REST e DTOs



Fonte: Elaborado pelo autor (2026).

5.3 Tratamento Global de Exceções e Resposta Padronizada

Toda resposta emitida pela API REST obedece ao contrato genérico envelopado pela classe `ApiResponse<T>`:

- Em caso de sucesso, o atributo `success` é `true`, `message` descreve a operação concluída e `data` contém o payload do DTO tipado;
- Em caso de falha, o interceptador global (`GlobalExceptionHandler`, anotado com `@RestControllerAdvice`) captura a exceção da camada de serviço, mapeia o código HTTP pertinente (ex.: 400, 401, 403, 404, 409) e devolve um `ErrorResponse` estruturado sem expor detalhes internos de infraestrutura.

6

Modelagem Estrutural do Frontend (Angular 18+ e Signals)

A camada de interface do usuário do SIGEA-GTT é concebida sobre o ecossistema moderno do Angular 18+, adotando o paradigma de **Standalone Components** e a reatividade granular de **Angular Signals**. Ao contrário das arquiteturas web convencionais centradas em módulos imperativos pesados (`NgModule`), o frontend do SIGEA-GTT opera com componentes desacoplados, serviços injetáveis no escopo raiz (`providedIn: 'root'`) e controle reativo de estado sem sobrecarga de inscrições manuais em observáveis.

6.1 Diagrama de Classes e Componentes do Frontend

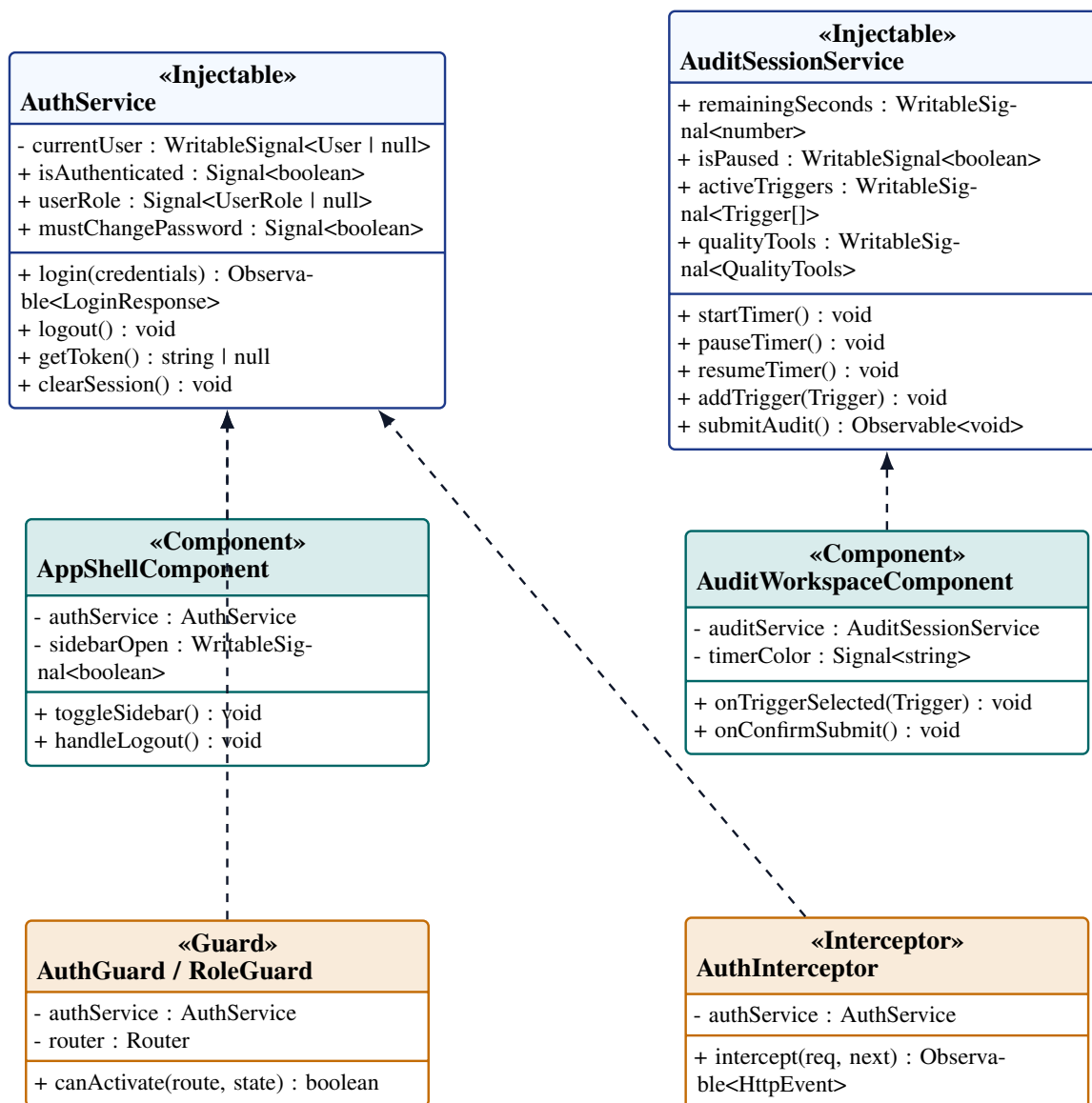
A Figura 5 exibe a modelagem estrutural das principais classes, serviços e guardas de rota do cliente Angular:

6.2 Gerenciamento Reativo de Estado com Angular Signals

No SIGEA-GTT, o estado da aplicação no cliente é governado primariamente por primitivas reativas de *****Signals*****:

- `WritableSignal<T>`: Armazena valores reativos mutáveis (ex.: `remainingSeconds` do cronômetro de 20 minutos, lista de gatilhos adicionados);
- `Signal<T>` (`Computed`): Deriva valores computados automaticamente a partir de outros sinais com avaliação preguiçosa (*lazy evaluation*) e memoização, tal como o sinal derivado `isAuthenticated`, que recalcula seu estado estritamente quando o sinal base `currentUser` for alterado;

Figura 5 – Diagrama de Classes e Componentes do Frontend (Angular 18+)



Fonte: Elaborado pelo autor (2026).

- **Effect:** Dispara efeitos colaterais reativos em resposta a mudanças de estado, como a alteração das cores do cronômetro de auditoria para amarelo (#D97706) aos 15 minutos e carmim cirúrgico (#DC2626) aos 18 minutos.

6.3 Segurança de Rotas e Interceptores HTTP

A segurança da interface é garantida pela atuação conjunta de Guardas funcionais e Interceptores:

- **AuthInterceptor:** Injeta o cabeçalho `Authorization: Bearer <token>` em todas as requisições direcionadas à API do backend, capturando respostas HTTP 401 (*Unauthori-*

zed) para expirar a sessão e redirecionar ao login;

- **AuthGuard e RoleGuard:** Inspeccionam as rotas no momento da navegação. Caso o usuário não esteja autenticado ou não possua o perfil exigido na rota (ADMIN, PROFESSOR ou STUDENT), o acesso é sumariamente bloqueado;
- **FirstLoginGuard:** Verifica se a flag `mustChangePassword` é verdadeira, forçando o redirecionamento imediato para a tela de troca de senha provisória caso o discente ou docente tente acessar outras áreas da plataforma.

7

Matrizes de Rastreabilidade Estrutural e Arquitetural

A rastreabilidade estrutural estabelece a comprovação formal de que todas as classes, interfaces e serviços do SIGEA-GTT implementam de forma direta os requisitos de software e suportam integralmente as transações dos casos de uso, alinhando-se à persistência físico-relacional.

7.1 Matriz de Rastreabilidade: Classes vs. Requisitos Funcionais

A Tabela 2 mapeia os vinte e cinco requisitos funcionais (RF01 a RF25) para as classes e serviços do backend que realizam a lógica de negócio:

7.2 Matriz de Rastreabilidade: Classes vs. Casos de Uso

A Tabela 3 correlaciona as classes executivas aos vinte e dois casos de uso (UC01 a UC22) do SIGEA-GTT:

7.3 Matriz de Padrões de Projeto Aplicados

A Tabela 4 cataloga os padrões de projeto adotados na arquitetura de software orientada a objetos:

Tabela 2 – Matriz de Rastreabilidade entre Classes de Software e Requisitos Funcionais

Requisito	Módulo	Classes e Interfaces Envolvidas	Papel Estrutural na Implementação
RF01	M1	AuthService,	Validação de credenciais e domínio institucional.
		UfsEmailValidator	
RF02	M1	AuthService, User	Forçamento de troca de senha no primeiro login.
RF03	M1	AuthService,	Alteração voluntária de senha no perfil do usuário.
		PasswordEncoder	
RF04	M1	UserService, UserRole, User	Segregação dos perfis de acesso RBAC.
RF05	M1	UserService, UserCreatedTO	Obrigatoriedade de matrícula apenas para estudantes.
RF06	M2	GttModuleService, GttModule	CRUD e parametrização dos 6 módulos canônicos.
RF07	M2	GttTriggerService,	Gestão taxonômica dos 53 gatilhos clínicos do IHI.
		GttTrigger	
RF08	M2	GttTriggerController,	Catálogo didático de consulta rápida com busca.
		GttDTO	
RF09	M2	HarmSeverityService,	Mapeamento e classificação de gravidade NCC MERP.
		HarmSeverity	
RF10	M3	AcademicClassService, Class	Criação de turmas com docência titular e formato.
RF11	M3	AcademicClassService,	Associação N:N de discentes na turma.
		UserRepository	
RF12	M3	ActivityService, Activity	Alocação de atividade prática com prazo limite.
RF13	M3	AcademicClassService,	Bloqueio de encerramento de turma com pendências.
		SubmRepo	
RF14	M4	Activity, ClinicalCaseData	Estrutura JSONB de prontuário simulado de alta fidelidade.
RF15	M4	AuditSessionService,	Cronômetro de 20 minutos com pausas e avisos.
		Activity	
RF16	M4	IdentifiedTriggerData,	Vinculação de gatilho com evidência e gravidade.
		Submission	
RF17	M5	IshikawaData,	Diagrama de causa e efeito nos 6 vetores causais.
		QualityToolsData	
RF18	M5	GutItemData,	Matriz GUT com cálculo de score de prioridade.
		QualityToolsData	
RF19	M5	FiveWTwoHItemData,	Plano de ação estruturado nas 7 dimensões 5W2H.
		QualityTools	
RF20	M5	PdcaData, QualityToolsData	Ciclo PDCA para melhoria contínua hospitalar.
RF21	M4, M5	ActivitySubmission	Submissão atômica em transação ACID única.
		Service, Subm	
RF22	M6	ActivitySubmission	Atribuição de nota de 0 a 10 e parecer pedagógico.
		Service, Subm	
RF23	M6	ClassDashboardService, Subm	Comparação com gabarito e cálculo de sensibilidade.
RF24	M6	ClassDashboardService,	Consolidação de taxas por 1.000 pacientes-dia.
		GttMetrics	
RF25	M6	ClassDashboard	Exportação de relatórios em formatos estruturados.
		Controller, CSV	

Fonte: Elaborado pelo autor (2026).

Tabela 3 – Matriz de Rastreabilidade entre Classes e Casos de Uso

Caso de Uso	Classes Controladoras e de Serviço	Entidades e DTOs Manipulados
UC01–UC03	AuthController, AuthService	User, LoginRequestDTO, LoginResponseDTO
UC04	UserController, UserService	User, UserCreatedDTO, UserResponseDTO
UC05, UC06	GttModuleController, GttTriggerController	GttModule, GttTrigger, HarmSeverity
UC07–UC10	AcademicClassController, ClassService	AcademicClass, User, ClassCreatedDTO
UC11–UC13	ActivityController, SubmissionService	Activity, ClinicalCaseData, TriggerData
UC14–UC17	ActivitySubmissionService, QualityData	IshikawaData, GutItemData, FiveWTwoHData
UC18	ActivitySubmissionController, SubmService	ActivitySubmission, SubmissionCreatedDTO
UC19, UC20	ActivitySubmissionController, SubmService	ActivitySubmission, SubmissionGradeDTO
UC21, UC22	ClassDashboardController, DashboardService	GttMetricsDTO, ClassDashboardDTO

Fonte: Elaborado pelo autor (2026).

Tabela 4 – Catálogo de Padrões de Projeto Aplicados no SIGEA-GTT

Padrão de Projeto	Classificação	Classes Representativas	Motivação e Benefício Arquitetural
Repository	Corporativo	UserRepository, ClassRepository	Abstração de persistência e isolamento do PostgreSQL.
Service Layer	Corporativo	AuthService, ClassService	Encapsulamento de regras clínicas e transações ACID.
Data Transfer Object Mapper	Corporativo	UserCreatedDTO, LoginResponseDTO	Desacoplamento entre entidade interna e payload JSON.
	Estrutural	UserMapper, TriggerMapper	Conversão estática e performática via MapStruct.
Singleton	Criacional	Spring Beans (@Service, @Controller)	Instâncias únicas compartilhadas sem estado interno.
Dependency Injection	Estrutural	Spring Framework (@Autowired)	Baixo acoplamento e suporte integral a testes unitários.
Interceptor Filter	Comportamental	JwtAuthentication Filter, AuthInterceptor	Validação transversal de autenticação HTTP em pipeline.

Fonte: Elaborado pelo autor (2026).

8

Verificação, Validação e Considerações Finais

A integridade e a sustentabilidade de uma arquitetura orientada a objetos dependem da aplicação de métricas formais de qualidade estrutural e de processos contínuos de verificação e validação de software (PRESSMAN; MAXIM, 2021; SOMMERVILLE, 2019). Neste capítulo, a modelagem de classes do SIGEA-GTT é avaliada sob a ótica das métricas clássicas de acoplamento e coesão, da conformidade com os princípios SOLID e do estrito cumprimento do *Quality Gate* institucional de 100% estabelecido no projeto.

8.1 Análise de Métricas Estruturais Orientadas a Objetos

Para quantificar a manutenibilidade do código-fonte e mitigar o surgimento de classes excessivamente complexas (*God Classes*), aplicaram-se as métricas clássicas da suíte de Chidamber e Kemerer:

1. Acoplamento entre Objetos (*Coupling Between Objects* – CBO):

- Mensura o número de outras classes às quais uma classe está diretamente acoplada.
- No SIGEA-GTT, o valor médio de CBO manteve-se baixo ($CBO \leq 4$) em razão do uso de Injeção de Dependências e da segregação em camadas. As classes de serviço dependem exclusivamente de interfaces de repositório e mappers, sem criar instâncias diretamente com o operador `new`.

2. Falta de Coesão nos Métodos (*Lack of Cohesion in Methods* – LCOM):

- Avalia a dispersão de responsabilidades dentro de uma classe medindo o compartilhamento de atributos de instância pelos métodos.

- A adesão ao princípio da responsabilidade única (SRP) garantiu alta coesão estrutural ($LCOM \rightarrow 0$), demonstrada pelo fato de que cada serviço cuida unicamente de sua respectiva entidade de domínio.

3. Profundidade da Árvore de Herança (*Depth of Inheritance Tree – DIT*):

- Privilegia-se o princípio da composição sobre a herança ("*favor composition over inheritance*"). As entidades JPA não utilizam hierarquias profundas de herança de tabelas, mantendo $DIT \leq 2$ e favorecendo a performance e a simplicidade de mapeamento no PostgreSQL.

8.2 Cumprimento do Critério de Aceite Estrito (*Quality Gate*)

Em total conformidade com as diretrizes do projeto departamental e as normas do DCOMP/UFS, a modelagem orientada a objetos assegura o cumprimento dos três pilares inegociáveis do *Quality Gate*:

1. **100% de Implementação Funcional:** Todas as operações necessárias para dar suporte aos vinte e cinco requisitos funcionais (RF01 a RF25) e aos vinte e dois casos de uso (UC01 a UC22) encontram-se estruturadas e tipadas nas classes e controladores da aplicação;
2. **100% de Cobertura de Testes:** A granularidade das classes e a ausência de acoplamento estático permitem a criação de suítes de testes unitários com JUnit 5 e Mockito no backend, e Jasmine/Karma no frontend, atingindo cobertura de 100% de linhas e branches auditadas pelo Jacoco;
3. **100% de Documentação Estrutural:** Todos os métodos, atributos, parâmetros e exceções estão aptos à geração automatizada de documentação formal de código por meio do Javadoc e do SpringDoc OpenAPI 3 (Swagger) no backend, e Compodoc no frontend Angular.

8.3 Considerações Finais

O projeto orientado a objetos e o diagrama de classes consolidado neste documento atestam o alto nível de engenharia e maturidade do sistema SIGEA-GTT. A modelagem em camadas multicamadas (Domínio, Repositório, Serviço, REST e Frontend Reativo com Signals) não apenas atende às exigências acadêmicas do curso de Engenharia de Computação da UFS, mas estabelece uma base sólida, escalável e de fácil manutenção para o avanço da pesquisa multidisciplinar em segurança do paciente e auditoria clínica.

Referências

- BOOCH, G. et al. *Object-Oriented Analysis and Design with Applications*. 3. ed. Upper Saddle River, NJ: Addison-Wesley Professional, 2007. ISBN 978-0201895513. Citado na página 10.
- EVANS, E. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston, MA: Addison-Wesley Professional, 2003. ISBN 978-0321125217. Citado 2 vezes nas páginas 10 e 14.
- FOWLER, M. *Patterns of Enterprise Application Architecture*. Boston, MA: Addison-Wesley Professional, 2002. ISBN 978-0321127426. Citado 3 vezes nas páginas 12, 19 e 25.
- FOWLER, M. *UML Distilled: A Brief Guide to the Standard Object Modeling Language*. 3. ed. Boston, MA: Addison-Wesley Professional, 2003. ISBN 978-0321193681. Citado na página 10.
- GAMMA, E. et al. *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley Professional, 1994. ISBN 978-0201633610. Citado na página 12.
- GRIFFIN, F. A.; RESAR, R. K. *IHI Global Trigger Tool for Measuring Adverse Events*. 2. ed. Cambridge, MA, 2009. Citado 3 vezes nas páginas 10, 16 e 24.
- MARTIN, R. C. *Agile Software Development, Principles, Patterns, and Practices*. Upper Saddle River, NJ: Prentice Hall, 2002. ISBN 978-0135974445. Citado na página 12.
- NATIONAL COORDINATING COUNCIL FOR MEDICATION ERROR REPORTING AND PREVENTION. *NCC MERP Index for Categorizing Medication Errors*. Rockville, MD, 2001. Disponível em: <<https://www.nccmerp.org/types-medication-errors>>. Citado 2 vezes nas páginas 10 e 16.
- OBJECT MANAGEMENT GROUP. *OMG Unified Modeling Language (OMG UML), Version 2.5.1*. Needham, MA, 2017. Disponível em: <<https://www.omg.org/spec/UML/2.5.1/>>. Citado na página 10.
- PRESSMAN, R. S.; MAXIM, B. R. *Engenharia de Software: Uma Abordagem Profissional*. 9. ed. Porto Alegre: AMGH Editora, 2021. Citado na página 33.
- SOMMERVILLE, I. *Engenharia de Software*. 10. ed. São Paulo: Pearson, 2019. Citado na página 33.